Edge AI and TinyML

Deploying Intelligence on Constrained Devices
CSE 521: Embedded Machine Learning

Omar Tahon
ID: 022025001

May 19, 2026

Outline

What is Edge AI?

Edge AI moves machine learning inference from centralized cloud servers directly to local hardware devices.

- **Why move to the Edge?**
  - **Latency:** Eliminate round-trip delays for real-time applications (e.g., autonomous driving).
  - **Bandwidth:** Avoid streaming massive amounts of raw data (like video feeds).
  - **Privacy:** Sensitive data remains on-device.
- **What is TinyML?**
  - A subset of Edge AI focused on deploying models on ultra-low power Microcontrollers (MCUs) running on milliwatts of power.

Edge Vision Models

Recent innovations deliver high accuracy with minimal computational overhead.

YOLOv11 (Nov 2024)

- Introduces transformer-based backbones with Partial Self-Attention (PSA).
- **Performance:** 25-40% lower latency than YOLOv10 with 60+ FPS on edge devices.

MobileNetV4

- Developed by Google for the mobile ecosystem.
- Uses Universal Inverted Bottleneck (UIB) blocks.
- **Performance:** 87% ImageNet accuracy at 3.8ms latency (Pixel 8 EdgeTPU).

Small Language Models (SLMs)

LLMs (like GPT-4) are too massive for the edge. The industry is building highly optimized **SLMs**:

Model	Parameters	Target Platform
Microsoft Phi-3	3.8B	CPU, GPU, Mobile
TinyLlama	1.1B	Mobile/Edge Devices
Google Gemini Nano	1.8B / 3.25B	Smartphones
Meta Llama 3.2	Varying	Vision & Edge

Table: Overview of State-of-the-Art SLMs.

Model Quantization

Problem: Floating-point (FP32) weights consume too much memory and power for edge devices.

Solution: Quantization maps high-precision floats to lower precision integers (INT8, INT4).

- **Post-Training Quantization (PTQ):** Convert after training. Fast, but can lose accuracy.
- **Quantization-Aware Training (QAT):** Simulate low precision during training. Recovers lost accuracy.
- **AWQ (Activation-aware Weight Quantization):** Only quantizes non-salient (non-critical) weights, allowing 4-bit precision with minimal accuracy loss.

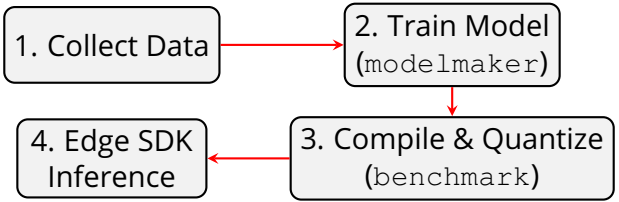
Code Example: INT8 Quantization

```python
import numpy as np

def quantize_int8(weights):
    min_val, max_val = np.min(weights), np.max(weights)

    # Calculate scale and zero-point for INT8 [-128, 127]
    scale = (max_val - min_val) / 255.0
    zero_point = -128 - np.round(min_val / scale)

    # Quantize and clip
    q_weights = np.round(weights / scale + zero_point)
    q_weights = np.clip(q_weights, -128, 127).astype(np.int8)

    return q_weights, scale, zero_point
```

Result: A 4.0x compression ratio (FP32 → INT8) reducing memory and increasing inference speed.

Deployment Workflow: `edgeai-tensorlab`

Using the Texas Instruments ecosystem as an example for end-to-end deployment:



- **edgeai-modelmaker:** Command-line tool for Bring-Your-Own-Data (BYOD) training.
- **edgeai-benchmark:** Handles Post-Training Quantization and generates hardware-specific executable artifacts.

Summary

- **Edge AI & TinyML** bring intelligence directly to hardware, ensuring low latency and high privacy.
- Architectures like **YOLOv11** and **MobileNetV4** push the boundaries of vision at the edge.
- **Quantization** (INT8/INT4) is critical for shrinking models to fit MCUs and NPUs.
- Ecosystems like **TI** `edgeai-tensorlab` abstract the complex compilation process, enabling rapid deployment.

Thank You!

Any Questions?